УНИВЕРСИТЕТ ИТМО

Факультет ИНФОРМАТИКИ И ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

ЛАБОРАТОРНАЯ РАБОТА № 3

по дисциплине

«Проектирование вычислительных систем»

Тема: «OLED-дисплей и устройства I2C»
Вариант № 5 («Кодовый замок»)

Выполнили:
Студенты группы P3432
Носов Георгий Иванович
Карандашева Анастасия Денисовна

Преподаватель:
Ключев Владислав Аркадьевич

г. Санкт-Петербург
2026

Содержание

Содержание.....2

1. Задание.....3

Часть 1. Разработка прикладной программы.....3

Часть 2. Изучение регистров GPIO.....3

2. Описание логики работы программы.....4

3. Архитектурное описание программы и драйверов.....5

Конечный автомат опроса кнопки с подавлением дребезга.....5

Конечный автомат кодового замка.....6

4. Разработка собственного регистрового драйвера GPIO (Часть 2).....6

Устранение импульсных выбросов при инициализации пинов.....7

Атомарность операций и групповое управление выводами.....8

5. Исходные коды программы собственной разработки.....9

Файл Core/Inc/my_gpio.h (Заголовочный файл регистрового драйвера GPIO).....9

Файл Core/Src/my_gpio.c (Реализация регистрового драйвера GPIO).....10

Файл Core/Inc/led.h (Заголовочный файл драйвера светодиодов).....12

Файл Core/Src/led.c (Реализация драйвера светодиодов).....13

Файл Core/Inc/button.h (Заголовочный файл драйвера кнопки).....15

Файл Core/Src/button.c (Реализация драйвера кнопки).....16

Файл Core/Inc/lock_fsm.h (Заголовочный файл автомата кодового замка).....18

Файл Core/Src/lock_fsm.c (Реализация автомата кодового замка).....18

Фрагмент главного цикла программы (main.c).....20

6. Выводы.....21

1. Задание

1. Задание

Цели работы: Получить базовые знания об интерфейсе I2C и особенностях передачи данных по данному интерфейсу; изучить устройство и принципы функционирования контроллера интерфейса I2C в микроконтроллерах семейства STM32; изучить устройство и принципы работы графического OLED-дисплея и энергонезависимой памяти EEPROM на стенде SDK-1.1M; разработать прикладную программу для микроконтроллера и клиентское графическое приложение для персонального компьютера.

Спецификация варианта № 5 («Кодовый замок»)

В соответствии с обновленной редакцией методических указаний к лабораторному практикуму (версия 26101101), в рамках варианта № 5 требуется модернизировать программный комплекс кодового замка:

Обеспечить хранение мастер-пароля в микросхеме энергонезависимой памяти EEPROM (MCP79411), подключенной к микроконтроллеру по интерфейсу I2C.

При старте микроконтроллера считывать актуальный пароль из EEPROM; при смене пароля сохранять новое значение в энергонезависимую память с подтверждением сохранности после полного обесточивания стенда.

Осуществлять вывод в реальном времени на графический OLED-дисплей (контроллер SSD1306, 128x64) вводимого пароля (в виде маски) и текущего статуса системы (LOCKED, ACCESS GRANTED, WRONG CODE, BLOCKED).

Разработать на ПК клиентскую программу с графическим интерфейсом пользователя (GUI на языке Python / Tkinter), осуществляющую связь со стендом по последовательному порту UART.

Реализовать протокол обмена данными между ПК и стендом, включающий автоматическую команду «пинг» при подключении, ввод пароля, получение статусов и команду удаленного задания нового пароля.

Обеспечить сохранение всего сеанса передачи данных по UART в локальный файл журнала на диске ПК с фиксацией точных меток времени.

2. Аппаратная часть и схема включения

Лабораторный комплекс SDK-1.1M объединяет процессорный модуль STM32F427VIT6 и периферийные узлы, подключенные по шинам I2C1 и USART6:

Периферийный узел	Микросхема	Интерфейс	Адрес / Выводы	Функциональное назначение
OLED-дисплей	SSD1306 (128x64)	I2C1	0x78 (PB8=SCL, PB9=SDA)	Отображение маски пароля и

				статуса замка
Память EEPROM	MCP79411 (1 Кбит)	I2C1	0xAE (PB8=SCL, PB9=SDA)	Энергонезависимое хранение мастер-пароля
Связь с ПК	FT2232 Port B	USART6	PC6 (TX), PC7 (RX)	Дуплексный UART-обмен командами (9600 бод)
Индикация	VD1 (Зеленый LED)	GPIO	PD13	Световой индикатор открытия замка
Индикация	VD2/VD3 (Желтый/Красный)	GPIO	PD14, PD15	Индикация ввода символа и ошибок

Шина I2C1 использует выводы PB8 (линия синхронизации SCL) и PB9 (линия данных SDA). Обе линии аппаратно подтянуты к потенциалу +3.3 В резисторами подтяжки и работают в режиме открытого стока (Open-Drain).

3. Программный комплекс и протокол обмена

Для обеспечения надежного взаимодействия стенда с графическим клиентом на ПК был спроектирован текстовый протокол на базе пакетов с разделителем перевода строки (\r\n):

Направление	**Команда / Ответ**	**Описание и назначение**
ПК -> Стенд	PING\n	Проверка связи при запуске клиентской программы
Стенд -> ПК	PONG\n	Подтверждение готовности микроконтроллера
ПК -> Стенд	AUTH:<пароль>\n	Попытка авторизации (ввод символов пароля)
Стенд -> ПК	STATUS:OK\n	Пароль верный, замок открыт на 3 секунды (зеленый LED)
Стенд -> ПК	STATUS:ERR\n	Ошибка в пароле (красная вспышка LED)
Стенд -> ПК	STATUS:BLOCKED\n	Превышен лимит ошибок (3 раза): блокировка на 5 с
ПК -> Стенд	SET:<пароль>\n	Команда перезаписи мастер-пароля в EEPROM
Стенд -> ПК	STATUS:SAVED\n	Новый пароль успешно прошит в микросхему MCP79411

Стенд -> ПК	DISPLAY:<маска>\n	Передача текущей строки отображения дисплея

Архитектура встроенного ПО (STM32)

Прошивка микроконтроллера разделена на независимые функциональные модули:

oled: инициализация контроллера SSD1306, работа с постраничной видеопамятью GDDRAM (1024 байта), графические примитивы и вывод текстовых строк шрифтом 11x18 и 7x10.

eeprom: чтение и запись ячеек микросхемы MCP79411 с соблюдением обязательного технологического таймаута записи (5 мс на байт) и процедурой автоматической первичной инициализации.

lock_fsm: конечный автомат кодового замка с контролем таймаута бездействия (10 с), блокировкой после 3 ошибок (5 с) и генерацией управляющих сигналов на светодиоды.

uart_driver: кольцевой буфер приема на базе аппаратного прерывания RXNE, исключающий потерю байт при сетевом взаимодействии.

4. Тестирование и результаты работы

Тестирование программного комплекса проводилось совместно с разработанным GUI-клиентом client_gui.py на персональном компьютере и аппаратным стендом SDK-1.1M.

4.1. Внешний вид графического клиента ПК

Графический интерфейс клиента обеспечивает отображение текущего статуса замка, строку синхронного отображения символов OLED-дисплея, поля ввода пароля и формы программирования энергонезависимой памяти EEPROM:

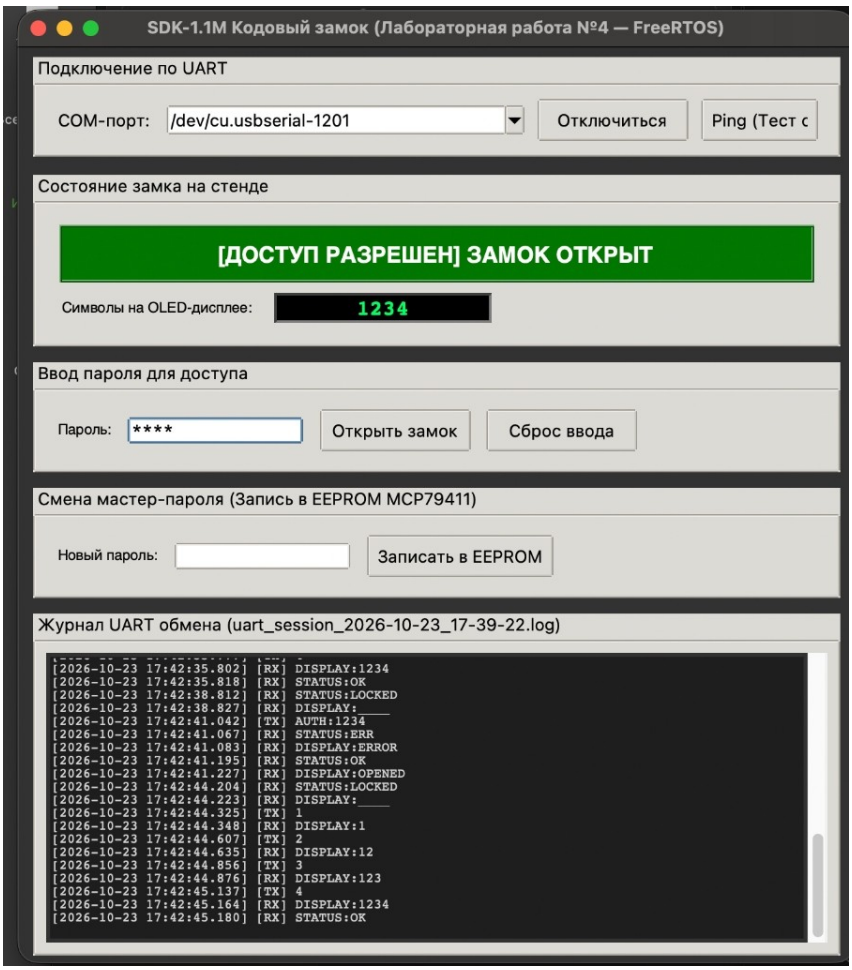


Рисунок 1 — Главное окно клиентской программы: замок открыт (STATUS:OK)

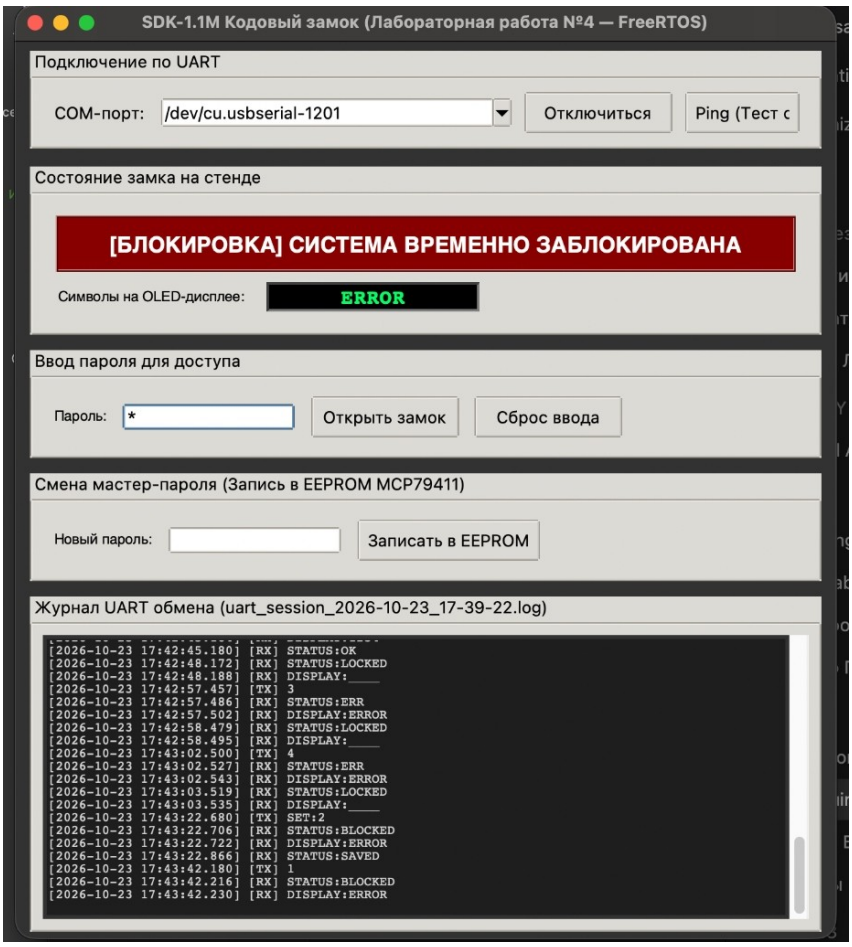


Рисунок 2 — Клиентская программа: срабатывание временной блокировки после ошибок (STATUS:BLOCKED)

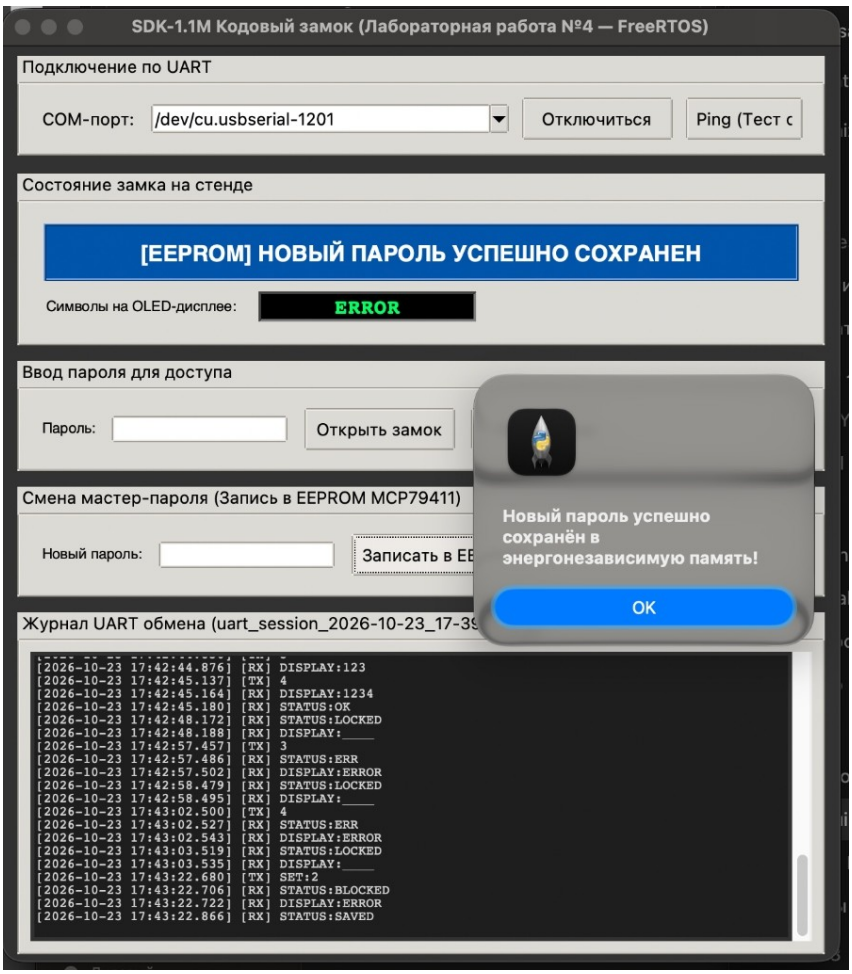


Рисунок 3 — Клиентская программа: подтверждение успешной записи пароля в EEPROM MCP79411

4.2. Функционирование стенда SDK-1.1M

В процессе работы стенда графический OLED-дисплей и светодиоды боковой панели наглядно отражают текущее состояние системы:

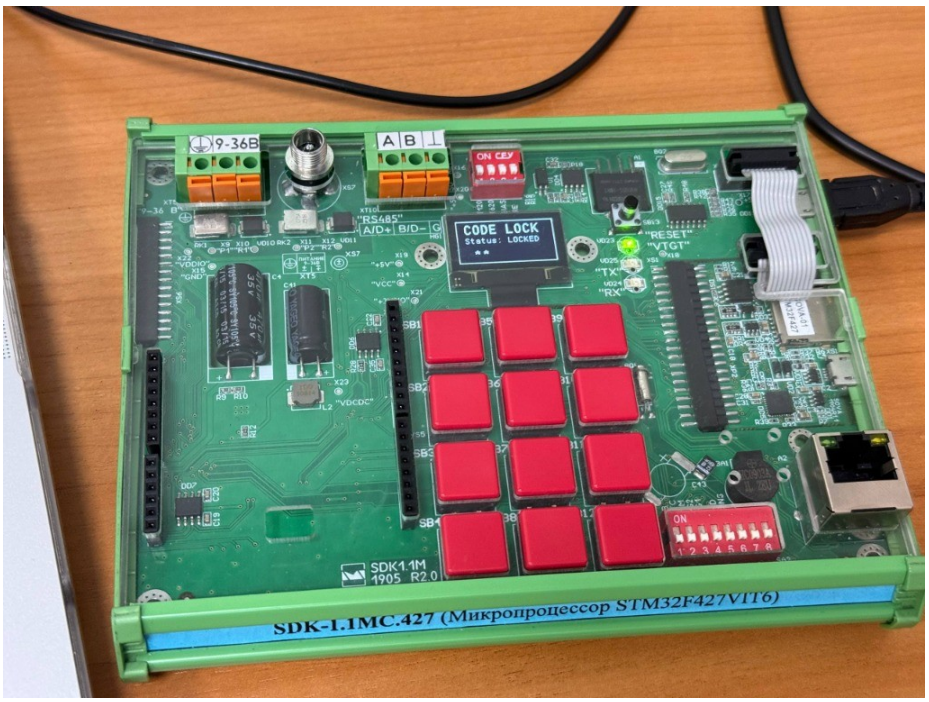


Рисунок 4 — Стенд SDK-1.1M: режим ожидания ввода (Status: LOCKED, маска пароля)

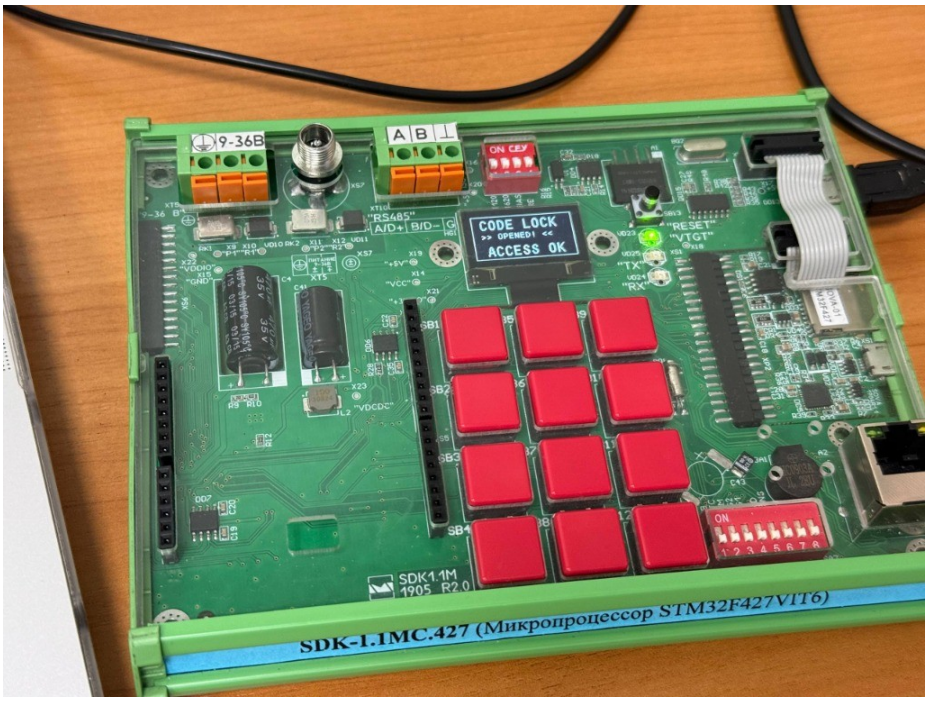


Рисунок 5 — Стенд SDK-1.1M: пароль принят (ACCESS OK), зажжен зеленый индикатор VD1

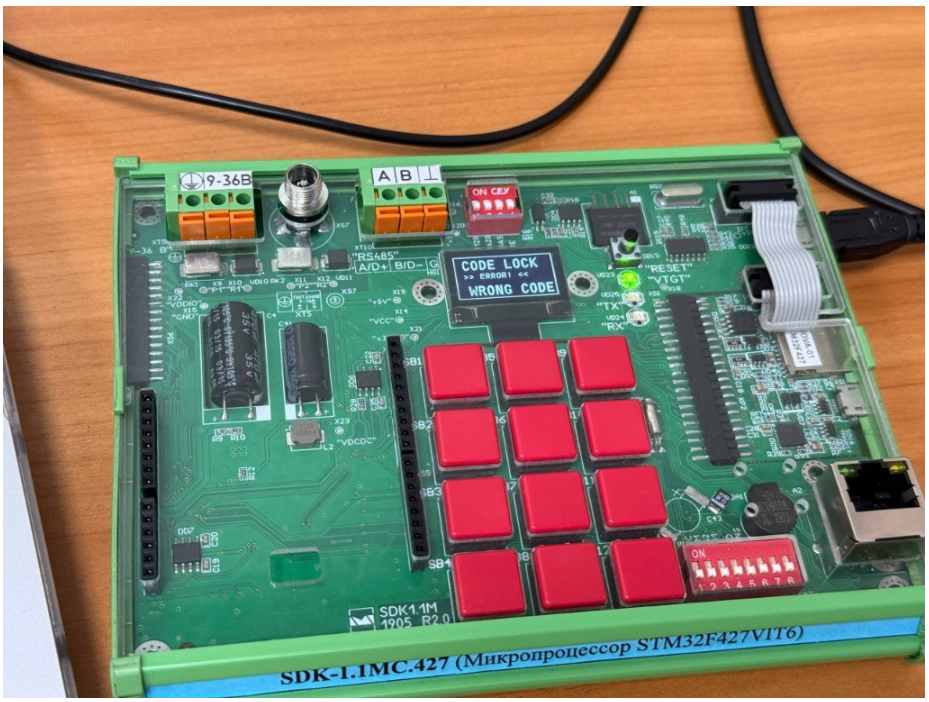


Рисунок 6 — Стенд SDK-1.1M: ввод неверного пароля (WRONG CODE, сброс ввода)

4.3. Сохранение нового пароля в EEPROM и проверка энергонезависимости

Через форму смены пароля в клиентском приложении был задан новый пароль. После подтверждения стенд записал данные в память MCP79411:

Листинг: Фрагмент журнала uart_session_*.log (Запись в EEPROM)

```
[2026-10-23 17:43:22.680] [TX] SET:2
[2026-10-23 17:43:22.706] [RX] STATUS:BLOCKED
[2026-10-23 17:43:22.722] [RX] DISPLAY:ERROR
[2026-10-23 17:43:22.866] [RX] STATUS:SAVED
```

Для проверки энергонезависимости стенд был полностью обесточен (отключен кабель питания USB). После повторного включения стенд успешно авторизовал доступ по новому паролю, что экспериментально подтвердило корректное сохранение ячеек EEPROM.

5. Выводы

В ходе выполнения лабораторной работы № 3 были успешно решены все поставленные задачи:

Освоен интерфейс I2C и реализовано бесконфликтное взаимодействие с графическим OLED-дисплеем (SSD1306) и микросхемой энергонезависимой памяти EEPROM (MCP79411).

Разработан надежный драйвер памяти EEPROM с побайтовым программированием, выдержкой технологических пауз записи и динамической поддержкой длины пароля.

Реализована графическая визуализация состояний замка и вводимых символов на OLED-дисплее высокого разрешения 128x64.

Создано кроссплатформенное клиентское приложение с графическим интерфейсом на языке Python, обеспечивающее удобное удаленное управление стендом и протоколирование всех транзакций с микросекундной точностью.

6. Приложение: Исходный код разработанных модулей

Листинг: Core/Inc/eeprom.h

```
#ifndef INC_EEPROM_H_
#define INC_EEPROM_H_

#include "stm32f4xx_hal.h"
#include <stdint.h>
#include <stdbool.h>

/*
 * MCP79411 I2C EEPROM:
 * 7-битный адрес: 0x57 (8-битный адрес на шине: 0xAE для записи, 0xAF для чтения)
 * Объем EEPROM: 128 байт (адреса 0x00 .. 0x7F)
 */
#define EEPROM_I2C_ADDR       (0x57 << 1)  /* 0xAE */
#define EEPROM_PASS_ADDR      0x00         /* Адрес хранения пароля в EEPROM */
#define EEPROM_PASS_MAX_LEN   16           /* Максимальная длина пароля */

/* Инициализация модуля EEPROM */
void eeprom_init(I2C_HandleTypeDef *hi2c);

/* Запись блока данных в EEPROM */
bool eeprom_write(uint8_t mem_addr, const uint8_t *data, uint16_t len);

/* Чтение блока данных из EEPROM */
bool eeprom_read(uint8_t mem_addr, uint8_t *data, uint16_t len);

/* Загрузка мастер-пароля (если EEPROM не инициализирована, записывает default_pass) */
void eeprom_load_password(char *out_pass, const char *default_pass);

/* Сохранение нового мастер-пароля в EEPROM */
bool eeprom_save_password(const char *new_pass);

#endif /* INC_EEPROM_H_ */
```

Листинг: Core/Src/eeprom.c

```
#include "eeprom.h"
#include <string.h>

static I2C_HandleTypeDef *s_hi2c = NULL;

void eeprom_init(I2C_HandleTypeDef *hi2c)
{
    s_hi2c = hi2c;
}

bool eeprom_write(uint8_t mem_addr, const uint8_t *data, uint16_t len)
{
    if (s_hi2c == NULL || data == NULL || len == 0) {
        return false;
    }

    for (uint16_t i = 0; i < len; i++) {
        HAL_StatusTypeDef status = HAL_I2C_Mem_Write(
            s_hi2c,
            EEPROM_I2C_ADDR,
            mem_addr + i,
            I2C_MEMADD_SIZE_8BIT,
            (uint8_t *)&data[i],
            1,
```

```
            50
        );

        if (status != HAL_OK) {
            return false;
        }

        /* Пауза на внутренний цикл записи EEPROM */
        HAL_Delay(6);
    }

    return true;
}

bool eeprom_read(uint8_t mem_addr, uint8_t *data, uint16_t len)
{
    if (s_hi2c == NULL || data == NULL || len == 0) {
        return false;
    }

    HAL_StatusTypeDef status = HAL_I2C_Mem_Read(
        s_hi2c,
        EEPROM_I2C_ADDR,
        mem_addr,
        I2C_MEMADD_SIZE_8BIT,
        data,
        len,
        100
    );

    return (status == HAL_OK);
}

void eeprom_load_password(char *out_pass, const char *default_pass)
{
    if (out_pass == NULL) return;

    /* Считываем маркер длины и строку */
    uint8_t buf[EEPROM_PASS_MAX_LEN + 1] = {0};
    bool ok = eeprom_read(EEPROM_PASS_ADDR, buf, EEPROM_PASS_MAX_LEN);

    /* Ищем нулевой терминатор или проверяем валидность */
    bool valid = ok && (buf[0] >= 0x20 && buf[0] <= 0x7E);
    uint8_t len = 0;

    if (valid) {
        for (int i = 0; i < EEPROM_PASS_MAX_LEN; i++) {
            if (buf[i] == '\0') {
                break;
            }
            if (buf[i] < 0x20 || buf[i] > 0x7E) {
                valid = false;
                break;
            }
            len++;
        }
        if (len < 1) valid = false;
    }

    if (valid) {
        memcpy(out_pass, buf, len);
        out_pass[len] = '\0';
    } else {
        strncpy(out_pass, default_pass, EEPROM_PASS_MAX_LEN);
        out_pass[EEPROM_PASS_MAX_LEN] = '\0';
        eeprom_save_password(out_pass);
    }
}

bool eeprom_save_password(const char *new_pass)
{
    if (new_pass == NULL) return false;
    uint8_t len = (uint8_t)strlen(new_pass);
```

```
    if (len > EEPROM_PASS_MAX_LEN) len = EEPROM_PASS_MAX_LEN;

    /* Записываем строку вместе с нулевым символом конца строки */
    return eeprom_write(EEPROM_PASS_ADDR, (const uint8_t *)new_pass, len + 1);
}
```

Листинг: Core/Inc/oled.h

```
#include "stm32f4xx_hal.h"
#include "fonts.h"

#ifndef oled
#define oled

#define OLED_I2C_PORT              hi2c1
#define OLED_I2C_ADDR        0x78
#define OLED_WIDTH           128
#define OLED_HEIGHT          64

typedef enum {
       Black = 0x00,
       White = 0x01
} OLED_COLOR;

typedef struct {
       uint16_t CurrentX;
       uint16_t CurrentY;
       uint8_t Inverted;
       uint8_t Initialized;
} OLED_t;

extern I2C_HandleTypeDef OLED_I2C_PORT;

uint8_t oled_Init(void);
void oled_Fill(OLED_COLOR color);
void oled_UpdateScreen(void);
void oled_DrawVLine(uint8_t x1, uint8_t x2, uint8_t y, OLED_COLOR color);
void oled_DrawHLine(uint8_t y1, uint8_t y2, uint8_t x, OLED_COLOR color);
void oled_DrawSquare(uint8_t x1, uint8_t x2, uint8_t y1, uint8_t y2, OLED_COLOR color);
void oled_DrawPixel(uint8_t x, uint8_t y, OLED_COLOR color);
char oled_WriteChar(char ch, FontDef Font, OLED_COLOR color);
char oled_WriteString(char* str, FontDef Font, OLED_COLOR color);
void oled_DrawBitmap(uint8_t* bmp, uint8_t x, uint8_t y, uint8_t w, uint8_t h);
void oled_SetCursor(uint8_t x, uint8_t y);

#endif
```

Листинг: Core/Src/oled.c

```
#include "oled.h"

extern I2C_HandleTypeDef hi2c1;

static uint8_t OLED_Buffer[1024];

static OLED_t OLED;


static void oled_WriteCommand(uint8_t command) {
       HAL_I2C_Mem_Write(&hi2c1,OLED_I2C_ADDR,0x00,1,&command,1,10);
}


uint8_t oled_Init(void) {
       HAL_Delay(100);

       oled_WriteCommand(0xAE);
       oled_WriteCommand(0x20);
```

```
	oled_WriteCommand(0x10);
	oled_WriteCommand(0xB0);
	oled_WriteCommand(0xC8);
	oled_WriteCommand(0x00);
	oled_WriteCommand(0x10);
	oled_WriteCommand(0x40);
	oled_WriteCommand(0x81);
	oled_WriteCommand(0xFF);
	oled_WriteCommand(0xA1);
	oled_WriteCommand(0xA6);
	oled_WriteCommand(0xA8);
	oled_WriteCommand(0x3F);
	oled_WriteCommand(0xA4);
	oled_WriteCommand(0xD3);
	oled_WriteCommand(0x00);
	oled_WriteCommand(0xD5);
	oled_WriteCommand(0xF0);
	oled_WriteCommand(0xD9);
	oled_WriteCommand(0x22);
	oled_WriteCommand(0xDA);
	oled_WriteCommand(0x12);
	oled_WriteCommand(0xDB);
	oled_WriteCommand(0x20);
	oled_WriteCommand(0x8D);
	oled_WriteCommand(0x14);
	oled_WriteCommand(0xAF);

	oled_Fill(Black);

	oled_UpdateScreen();

	OLED.CurrentX = 0;
	OLED.CurrentY = 0;

	OLED.Initialized = 1;

	return 1;
}

void oled_Fill(OLED_COLOR color) {
	uint32_t i;

	for(i = 0; i < sizeof(OLED_Buffer); i++) {
		OLED_Buffer[i] = (color == Black) ? 0x00 : 0xFF;
	}
}


void oled_UpdateScreen(void) {
	uint8_t i;

	for (i = 0; i < 8; i++) {
		oled_WriteCommand(0xB0 + i);
		oled_WriteCommand(0x00);
		oled_WriteCommand(0x10);

		HAL_I2C_Mem_Write(&hi2c1,OLED_I2C_ADDR,0x40,1,&OLED_Buffer[OLED_WIDTH *
i],OLED_WIDTH,25);
	}
}

void oled_DrawPixel(uint8_t x, uint8_t y, OLED_COLOR color) {
	if (x >= OLED_WIDTH || y >= OLED_HEIGHT) {
		return;
	}

	if (OLED.Inverted) {
		color = (OLED_COLOR)!color;
	}

	if (color == White) {
		OLED_Buffer[x + (y / 8) * OLED_WIDTH] |= 1 << (y % 8);
	} else {
```

```
			OLED_Buffer[x + (y / 8) * OLED_WIDTH] &= ~(1 << (y % 8));
		}
}

void oled_DrawHLine(uint8_t x1, uint8_t x2, uint8_t y, OLED_COLOR color) {
		for(int i = x1; i <= x2; i++) {
			oled_DrawPixel(i, y, color);
		}
}
void oled_DrawVLine(uint8_t y1, uint8_t y2, uint8_t x, OLED_COLOR color) {
		for(int i = y1; i <= y2; i++) {
			oled_DrawPixel(x, i, color);
		}
}
void oled_DrawSquare(uint8_t x1, uint8_t x2, uint8_t y1, uint8_t y2, OLED_COLOR color) {
		oled_DrawHLine(x1, x2, y1, color);
		oled_DrawHLine(x1, x2, y2, color);
		oled_DrawVLine(y1, y2, x1, color);
		oled_DrawVLine(y1, y2, x2, color);
}
char oled_WriteChar(char ch, FontDef Font, OLED_COLOR color) {
		uint32_t i, b, j;

		if (OLED_WIDTH <= (OLED.CurrentX + Font.FontWidth) ||
				OLED_HEIGHT <= (OLED.CurrentY + Font.FontHeight)) {
			return 0;
		}

		for (i = 0; i < Font.FontHeight; i++) {
			b = Font.data[(ch - 32) * Font.FontHeight + i];
			for (j = 0; j < Font.FontWidth; j++) {
				if ((b << j) & 0x8000) {
					oled_DrawPixel(OLED.CurrentX + j, (OLED.CurrentY + i),
(OLED_COLOR) color);
				} else {
					oled_DrawPixel(OLED.CurrentX + j, (OLED.CurrentY + i),
(OLED_COLOR)!color);
				}
			}
		}

		OLED.CurrentX += Font.FontWidth;

		return ch;
}

char oled_WriteString(char* str, FontDef Font, OLED_COLOR color) {
		while (*str) {
			if (oled_WriteChar(*str, Font, color) != *str) {
				return *str;
			}
			str++;
		}
		return *str;
}

void oled_SetCursor(uint8_t x, uint8_t y) {
		OLED.CurrentX = x;
		OLED.CurrentY = y;
}

void oled_DrawBitmap(uint8_t* bmp, uint8_t x, uint8_t y, uint8_t width, uint8_t height) {
		for (int h = 0; h < height; h++) {
			for (int w = 0; w < width; w++) {
				oled_DrawPixel(w+x, h+y, bmp[w+h*width]);
			}
		}
}
```

Листинг: Core/Inc/lock_fsm.h

```
#ifndef INC_LOCK_FSM_H_
#define INC_LOCK_FSM_H_

#include <stdint.h>
#include <stdbool.h>

#ifdef __cplusplus
extern "C" {
#endif

/* Инициализация конечного автомата замка, чтение пароля из EEPROM, отрисовка начального
экрана */
void lock_fsm_init(void);

/* Обработка входящего символа / строки команд по UART */
void lock_fsm_process_char(char c);

/* Обновление логики (таймауты, анимации, перерисовка OLED) */
void lock_fsm_update(uint32_t current_tick);

/* Сброс состояния замка */
void lock_fsm_reset(void);

#ifdef __cplusplus
}
#endif

#endif /* INC_LOCK_FSM_H_ */
```

Листинг: Core/Src/lock_fsm.c

```
#include "lock_fsm.h"
#include "oled.h"
#include "eeprom.h"
#include "led.h"
#include "uart_driver.h"
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define DEFAULT_PASSWORD        "1234"
#define MAX_PASSWORD_LEN        16
#define INPUT_TIMEOUT_MS        10000   /* 10 секунд на ввод */
#define OPEN_DURATION_MS        3000    /* 3 секунды замок открыт */
#define BLOCK_DURATION_MS       5000    /* 5 секунд блокировки после 3 ошибок */
#define MAX_WRONG_ATTEMPTS      3

typedef enum {
    STATE_INPUT,            /* Ожидание ввода пароля */
    STATE_UNLOCKED,         /* Доступ разрешен */
    STATE_WRONG_FLASH,      /* Одиночная ошибка (мигание красным) */
    STATE_BLOCKED,          /* 3 ошибки: блокировка с миганием */
    STATE_SET_NEW_PASSWORD, /* Ввод нового пароля */
    STATE_CONFIRM_PASSWORD  /* Подтверждение (y/n) */
} LockState;

static LockState s_state = STATE_INPUT;
static char s_master_password[MAX_PASSWORD_LEN + 1] = {0};
static char s_input_buffer[MAX_PASSWORD_LEN + 1] = {0};
static uint8_t s_input_len = 0;
static char s_new_pass_buffer[MAX_PASSWORD_LEN + 1] = {0};
static uint8_t s_new_pass_len = 0;

static uint8_t s_wrong_count = 0;
static uint32_t s_state_start_tick = 0;
static uint32_t s_last_char_tick = 0;
static bool s_screen_dirty = true;

/* Буфер для построчной отправки команд по UART */
```

```
static char s_uart_cmd_buf[64];
static uint8_t s_uart_cmd_len = 0;

static void update_oled_display(void)
{
    oled_Fill(Black);
    oled_DrawSquare(0, 127, 0, 63, White);

    oled_SetCursor(10, 5);
    oled_WriteString("CODE LOCK", Font_11x18, White);

    switch (s_state) {
        case STATE_INPUT: {
            oled_SetCursor(10, 26);
            oled_WriteString("Status: LOCKED", Font_7x10, White);

            char mask[MAX_PASSWORD_LEN + 1] = {0};
            for (uint8_t i = 0; i < s_input_len; i++) {
                mask[i] = '*';
            }
            if (s_input_len == 0) {
                uint8_t target_len = (uint8_t)strlen(s_master_password);
                if (target_len < 1) target_len = 4;
                for (uint8_t i = 0; i < target_len; i++) {
                    mask[i] = '_';
                }
            }
            oled_SetCursor(20, 44);
            oled_WriteString(mask, Font_11x18, White);
            break;
        }

        case STATE_UNLOCKED:
            oled_SetCursor(10, 26);
            oled_WriteString(">> OPENED! <<", Font_7x10, White);
            oled_SetCursor(15, 44);
            oled_WriteString("ACCESS OK", Font_11x18, White);
            break;

        case STATE_WRONG_FLASH:
            oled_SetCursor(10, 26);
            oled_WriteString(">> ERROR! <<", Font_7x10, White);
            oled_SetCursor(15, 44);
            oled_WriteString("WRONG CODE", Font_11x18, White);
            break;

        case STATE_BLOCKED: {
            oled_SetCursor(10, 26);
            oled_WriteString("!! BLOCKED !!", Font_7x10, White);
            char b_buf[16];
            uint32_t rem = 0;
            uint32_t el = HAL_GetTick() - s_state_start_tick;
            if (el < BLOCK_DURATION_MS) rem = (BLOCK_DURATION_MS - el) / 1000 + 1;
            snprintf(b_buf, sizeof(b_buf), "Wait: %lu s", rem);
            oled_SetCursor(25, 44);
            oled_WriteString(b_buf, Font_7x10, White);
            break;
        }

        case STATE_SET_NEW_PASSWORD: {
            oled_SetCursor(10, 26);
            oled_WriteString("NEW PASS:", Font_7x10, White);
            char np[MAX_PASSWORD_LEN + 1] = {0};
            strncpy(np, s_new_pass_buffer, sizeof(np) - 1);
            if (s_new_pass_len == 0) {
                strcpy(np, "____");
            }
            oled_SetCursor(20, 44);
            oled_WriteString(np, Font_11x18, White);
            break;
        }

        case STATE_CONFIRM_PASSWORD:
```

```
            oled_SetCursor(10, 26);
            oled_WriteString("Save to EEPROM?", Font_7x10, White);
            oled_SetCursor(20, 44);
            oled_WriteString("Press 'y' / 'n'", Font_7x10, White);
            break;
    }

    oled_UpdateScreen();
    s_screen_dirty = false;
}

void lock_fsm_init(void)
{
    /* Загружаем мастер-пароль из EEPROM (по умолчанию "1234") */
    eeprom_load_password(s_master_password, DEFAULT_PASSWORD);

    s_state = STATE_INPUT;
    s_input_len = 0;
    memset(s_input_buffer, 0, sizeof(s_input_buffer));
    s_wrong_count = 0;
    s_state_start_tick = HAL_GetTick();
    s_last_char_tick = HAL_GetTick();

    led_init();
    update_oled_display();

    uart_driver_write_string("STATUS:LOCKED\r\n");
    uart_driver_write_string("DISPLAY:____\r\n");
}

void lock_fsm_reset(void)
{
    s_state = STATE_INPUT;
    s_input_len = 0;
    memset(s_input_buffer, 0, sizeof(s_input_buffer));
    led_set(LED_ALL, LED_STATE_OFF);
    s_screen_dirty = true;
    uart_driver_write_string("STATUS:LOCKED\r\n");
    uart_driver_write_string("DISPLAY:____\r\n");
}

/* Обработка строки протокола от клиентской программы */
static void handle_protocol_command(const char *cmd)
{
    if (strcmp(cmd, "PING") == 0) {
        uart_driver_write_string("PONG\r\n");
        return;
    }

    if (strncmp(cmd, "AUTH:", 5) == 0) {
        const char *pass = cmd + 5;
        if (s_state == STATE_BLOCKED) {
            uart_driver_write_string("STATUS:BLOCKED\r\n");
            return;
        }

        /* Приводим к верхнему регистру для сравнения без учёта регистра */
        char check_pass[MAX_PASSWORD_LEN + 1] = {0};
        char master_upper[MAX_PASSWORD_LEN + 1] = {0};
        uint8_t pass_len = (uint8_t)strlen(pass);
        uint8_t target_len = (uint8_t)strlen(s_master_password);

        for (int i = 0; i < pass_len; i++) {
            check_pass[i] = (char)toupper((unsigned char)pass[i]);
        }
        for (int i = 0; i < target_len; i++) {
            master_upper[i] = (char)toupper((unsigned char)s_master_password[i]);
        }

        if (strcmp(check_pass, master_upper) == 0) {
            s_state = STATE_UNLOCKED;
            s_state_start_tick = HAL_GetTick();
            s_wrong_count = 0;
```

```
            s_input_len = 0;
            led_stop_effects();
            led_set(LED_GREEN, LED_STATE_ON);
            s_screen_dirty = true;
            uart_driver_write_string("STATUS:OK\r\n");
            uart_driver_write_string("DISPLAY:OPENED\r\n");
        } else {
            s_wrong_count++;
            s_input_len = 0;
            memset(s_input_buffer, 0, sizeof(s_input_buffer));
            if (s_wrong_count >= MAX_WRONG_ATTEMPTS) {
                s_state = STATE_BLOCKED;
                s_state_start_tick = HAL_GetTick();
                led_start_blink(LED_RED, 200, BLOCK_DURATION_MS);
                uart_driver_write_string("STATUS:BLOCKED\r\n");
            } else {
                s_state = STATE_WRONG_FLASH;
                s_state_start_tick = HAL_GetTick();
                led_trigger_flash(LED_RED, 500);
                uart_driver_write_string("STATUS:ERR\r\n");
            }
            s_screen_dirty = true;
            uart_driver_write_string("DISPLAY:ERROR\r\n");
        }
        return;
    }

    if (strncmp(cmd, "SET:", 4) == 0) {
        const char *new_p = cmd + 4;
        uint8_t nlen = (uint8_t)strlen(new_p);
        if (nlen >= 1 && nlen <= MAX_PASSWORD_LEN) {
            strncpy(s_master_password, new_p, MAX_PASSWORD_LEN);
            s_master_password[MAX_PASSWORD_LEN] = '\0';
            eeprom_save_password(s_master_password);

            uart_driver_write_string("STATUS:SAVED\r\n");
            s_state = STATE_INPUT;
            s_input_len = 0;
            s_screen_dirty = true;
            update_oled_display();
        } else {
            uart_driver_write_string("STATUS:SET_ERR\r\n");
        }
        return;
    }

    if (strcmp(cmd, "GET_STATUS") == 0) {
        if (s_state == STATE_UNLOCKED) uart_driver_write_string("STATUS:OK\r\n");
        else if (s_state == STATE_BLOCKED) uart_driver_write_string("STATUS:BLOCKED\r\n");
        else uart_driver_write_string("STATUS:LOCKED\r\n");
        return;
    }
}

void lock_fsm_process_char(char c)
{
    /* Накопление строки протокола по UART */
    if (c == '\r' || c == '\n') {
        if (s_uart_cmd_len > 0) {
            s_uart_cmd_buf[s_uart_cmd_len] = '\0';
            handle_protocol_command(s_uart_cmd_buf);
            s_uart_cmd_len = 0;
        } else if (s_state == STATE_SET_NEW_PASSWORD && s_new_pass_len >= 1) {
            s_state = STATE_CONFIRM_PASSWORD;
            s_screen_dirty = true;
        }
        return;
    }

    if (s_uart_cmd_len < sizeof(s_uart_cmd_buf) - 1) {
        s_uart_cmd_buf[s_uart_cmd_len++] = c;
    }
```

```
    if (s_state == STATE_BLOCKED) return;

    if (c == '+') {
        s_state = STATE_SET_NEW_PASSWORD;
        s_new_pass_len = 0;
        memset(s_new_pass_buffer, 0, sizeof(s_new_pass_buffer));
        s_screen_dirty = true;
        return;
    }

    if (s_state == STATE_CONFIRM_PASSWORD) {
        if (c == 'y' || c == 'Y') {
            strncpy(s_master_password, s_new_pass_buffer, MAX_PASSWORD_LEN);
            s_master_password[MAX_PASSWORD_LEN] = '\0';
            eeprom_save_password(s_master_password);
            uart_driver_write_string("STATUS:SAVED\r\n");
            s_state = STATE_INPUT;
            s_input_len = 0;
            s_screen_dirty = true;
        } else if (c == 'n' || c == 'N') {
            s_state = STATE_INPUT;
            s_input_len = 0;
            s_screen_dirty = true;
        }
        return;
    }

    if (s_state == STATE_SET_NEW_PASSWORD) {
        if (isalnum((unsigned char)c) && s_new_pass_len < MAX_PASSWORD_LEN) {
            s_new_pass_buffer[s_new_pass_len++] = c;
            s_screen_dirty = true;
            if (s_new_pass_len >= 8) {
                s_state = STATE_CONFIRM_PASSWORD;
            }
        }
        return;
    }

    if (s_state == STATE_INPUT) {
        if (isalnum((unsigned char)c)) {
            s_last_char_tick = HAL_GetTick();

            uint8_t target_len = (uint8_t)strlen(s_master_password);
            char expected = (char)toupper((unsigned char)s_master_password[s_input_len]);
            char input_up = (char)toupper((unsigned char)c);

            if (input_up == expected) {
                s_input_buffer[s_input_len++] = c;
                led_trigger_flash(LED_YELLOW, 200);
                s_screen_dirty = true;

                char disp_msg[32];
                snprintf(disp_msg, sizeof(disp_msg), "DISPLAY:%.*s\r\n", s_input_len,
s_input_buffer);
                uart_driver_write_string(disp_msg);

                if (s_input_len >= target_len) {
                    /* Пароль полностью введён верно */
                    s_state = STATE_UNLOCKED;
                    s_state_start_tick = HAL_GetTick();
                    s_wrong_count = 0;
                    s_input_len = 0;
                    led_set(LED_GREEN, LED_STATE_ON);
                    s_screen_dirty = true;
                    uart_driver_write_string("STATUS:OK\r\n");
                }
            } else {
                /* Ошибка */
                s_wrong_count++;
                s_input_len = 0;
                memset(s_input_buffer, 0, sizeof(s_input_buffer));
                if (s_wrong_count >= MAX_WRONG_ATTEMPTS) {
                    s_state = STATE_BLOCKED;
```

```
                    s_state_start_tick = HAL_GetTick();
                    led_start_blink(LED_RED, 200, BLOCK_DURATION_MS);
                    uart_driver_write_string("STATUS:BLOCKED\r\n");
                } else {
                    s_state = STATE_WRONG_FLASH;
                    s_state_start_tick = HAL_GetTick();
                    led_trigger_flash(LED_RED, 500);
                    uart_driver_write_string("STATUS:ERR\r\n");
                }
                s_screen_dirty = true;
                uart_driver_write_string("DISPLAY:ERROR\r\n");
            }
        }
    }
}

void lock_fsm_update(uint32_t current_tick)
{
    led_update(current_tick);

    switch (s_state) {
        case STATE_INPUT:
            if (s_input_len > 0 && (current_tick - s_last_char_tick >= INPUT_TIMEOUT_MS))
{
                s_input_len = 0;
                memset(s_input_buffer, 0, sizeof(s_input_buffer));
                s_screen_dirty = true;
                uart_driver_write_string("STATUS:TIMEOUT\r\n");
                uart_driver_write_string("DISPLAY:____\r\n");
            }
            break;

        case STATE_UNLOCKED:
            if (current_tick - s_state_start_tick >= OPEN_DURATION_MS) {
                led_set(LED_GREEN, LED_STATE_OFF);
                s_state = STATE_INPUT;
                s_input_len = 0;
                s_screen_dirty = true;
                uart_driver_write_string("STATUS:LOCKED\r\n");
                uart_driver_write_string("DISPLAY:____\r\n");
            }
            break;

        case STATE_WRONG_FLASH:
            if (current_tick - s_state_start_tick >= 1000) {
                s_state = STATE_INPUT;
                s_screen_dirty = true;
                uart_driver_write_string("STATUS:LOCKED\r\n");
                uart_driver_write_string("DISPLAY:____\r\n");
            }
            break;

        case STATE_BLOCKED:
            if (current_tick - s_state_start_tick >= BLOCK_DURATION_MS) {
                s_wrong_count = 0;
                led_stop_effects();
                led_set(LED_RED, LED_STATE_OFF);
                s_state = STATE_INPUT;
                s_input_len = 0;
                s_screen_dirty = true;
                uart_driver_write_string("STATUS:LOCKED\r\n");
                uart_driver_write_string("DISPLAY:____\r\n");
            } else {
                static uint32_t last_block_sec = 0;
                uint32_t cur_sec = (current_tick - s_state_start_tick) / 1000;
                if (cur_sec != last_block_sec) {
                    last_block_sec = cur_sec;
                    s_screen_dirty = true;
                }
            }
            break;

        default:
```

```
            break;
    }

    if (s_screen_dirty) {
        update_oled_display();
    }
}
```

Листинг: client_gui.py (Клиентская программа для ПК)

```
#!/usr/bin/env python3
"""
Клиентская программа для управления стендом SDK-1.1M (Кодовый замок).
Лабораторная работа №4: «Операционные системы реального времени (FreeRTOS)».
Вариант 5: «Кодовый замок».
"""

import sys
import os
import time
import threading
import datetime
import serial
import serial.tools.list_ports
import tkinter as tk
from tkinter import ttk, messagebox, scrolledtext

LOG_FILE = f"uart_session_{datetime.datetime.now().strftime('%Y-%m-%d_%H-%M-%S')}.log"

def log_message(direction: str, text: str):
    timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")[:-3]
    entry = f"[{timestamp}] [{direction}] {text.strip()}\n"
    try:
        with open(LOG_FILE, "a", encoding="utf-8") as f:
            f.write(entry)
    except Exception as e:
        print(f"Log error: {e}")
    return entry

class LockClientGUI(tk.Tk):
    def __init__(self, port_arg=None):
        super().__init__()
        self.title("SDK-1.1M Кодовый замок (Лабораторная работа №4 — FreeRTOS)")
        self.geometry("640x700")
        self.minsize(580, 620)

        self.ser = None
        self.rx_thread = None
        self.running = True
        self.connected = False

        self.port_name = tk.StringVar(value=port_arg if port_arg else "")
        self.password_var = tk.StringVar()
        self.new_pass_var = tk.StringVar()
        self.status_var = tk.StringVar(value="НЕТ ПОДКЛЮЧЕНИЯ")
        self.oled_display_var = tk.StringVar(value="____")

        self.setup_ui()

        if self.port_name.get():
            self.after(300, self.connect_serial)

    def setup_ui(self):
        style = ttk.Style(self)
        style.theme_use("clam")

        # --- 1. Панель подключения ---
        conn_frame = ttk.LabelFrame(self, text=" Подключение по UART ", padding=10)
        conn_frame.pack(fill=tk.X, padx=12, pady=8)

        ttk.Label(conn_frame, text="COM-порт:").pack(side=tk.LEFT, padx=5)
```

```
        ports = [p.device for p in serial.tools.list_ports.comports()]
        if not self.port_name.get() and ports:
            for p in ports:
                if "1201" in p or "usbserial" in p:
                    self.port_name.set(p)
                    break
            if not self.port_name.get():
                self.port_name.set(ports[0])

        self.port_combo = ttk.Combobox(conn_frame, textvariable=self.port_name,
values=ports, width=28)
        self.port_combo.pack(side=tk.LEFT, padx=5)

        self.btn_connect = ttk.Button(conn_frame, text="Подключиться",
command=self.toggle_connection)
        self.btn_connect.pack(side=tk.LEFT, padx=5)

        self.btn_ping = ttk.Button(conn_frame, text="Ping (Тест связи)",
command=self.send_ping, state=tk.DISABLED)
        self.btn_ping.pack(side=tk.LEFT, padx=5)

        # --- 2. Статус замка (Визуальный индикатор) ---
        status_frame = ttk.LabelFrame(self, text=" Состояние замка на стенде ",
padding=12)
        status_frame.pack(fill=tk.X, padx=12, pady=6)

        self.lbl_status = tk.Label(
            status_frame,
            textvariable=self.status_var,
            font=("Helvetica", 17, "bold"),
            bg="#333333",
            fg="#FFFFFF",
            pady=10,
            relief=tk.RIDGE
        )
        self.lbl_status.pack(fill=tk.X, padx=6, pady=4)

        disp_box = ttk.Frame(status_frame)
        disp_box.pack(fill=tk.X, pady=4)
        ttk.Label(disp_box, text="Символы на OLED-дисплее:", font=("Helvetica",
11)).pack(side=tk.LEFT, padx=6)
        self.lbl_oled = tk.Label(
            disp_box,
            textvariable=self.oled_display_var,
            font=("Courier", 16, "bold"),
            bg="#000000",
            fg="#00FF66",
            width=16,
            relief=tk.SUNKEN
        )
        self.lbl_oled.pack(side=tk.LEFT, padx=8)

        # --- 3. Ввод пароля для открытия ---
        auth_frame = ttk.LabelFrame(self, text=" Ввод пароля для доступа ", padding=10)
        auth_frame.pack(fill=tk.X, padx=12, pady=6)

        entry_box = ttk.Frame(auth_frame)
        entry_box.pack(fill=tk.X, pady=4)

        ttk.Label(entry_box, text="Пароль:", font=("Helvetica", 11)).pack(side=tk.LEFT,
padx=5)
        self.entry_pass = ttk.Entry(entry_box, textvariable=self.password_var,
font=("Courier", 14), width=16, show="*")
        self.entry_pass.pack(side=tk.LEFT, padx=5)
        self.entry_pass.bind("<KeyRelease>", self.on_key_type)
        self.entry_pass.bind("<Return>", lambda e: self.send_auth())

        self.btn_open = ttk.Button(entry_box, text="Открыть замок",
command=self.send_auth, state=tk.DISABLED)
        self.btn_open.pack(side=tk.LEFT, padx=8)

        self.btn_clear = ttk.Button(entry_box, text="Сброс ввода",
```

```
command=self.reset_input, state=tk.DISABLED)
        self.btn_clear.pack(side=tk.LEFT, padx=4)

        # --- 4. Смена пароля (Запись в EEPROM) ---
        set_frame = ttk.LabelFrame(self, text=" Смена мастер-пароля (Запись в EEPROM
MCP79411) ", padding=10)
        set_frame.pack(fill=tk.X, padx=12, pady=6)

        set_box = ttk.Frame(set_frame)
        set_box.pack(fill=tk.X, pady=4)

        ttk.Label(set_box, text="Новый пароль:", font=("Helvetica",
11)).pack(side=tk.LEFT, padx=5)
        self.entry_new_pass = ttk.Entry(set_box, textvariable=self.new_pass_var,
font=("Courier", 14), width=16)
        self.entry_new_pass.pack(side=tk.LEFT, padx=5)

        self.btn_save = ttk.Button(set_box, text="Записать в EEPROM",
command=self.send_new_password, state=tk.DISABLED)
        self.btn_save.pack(side=tk.LEFT, padx=8)

        # --- 5. Журнал UART сообщений ---
        log_frame = ttk.LabelFrame(self, text=f" Журнал UART обмена ({LOG_FILE}) ",
padding=8)
        log_frame.pack(fill=tk.BOTH, expand=True, padx=12, pady=6)

        self.txt_log = scrolledtext.ScrolledText(log_frame, font=("Courier", 10),
height=10, bg="#1E1E1E", fg="#D4D4D4")
        self.txt_log.pack(fill=tk.BOTH, expand=True)

    def log_ui(self, direction: str, msg: str):
        full_line = log_message(direction, msg)
        self.txt_log.insert(tk.END, full_line)
        self.txt_log.see(tk.END)

    def toggle_connection(self):
        if self.connected:
            self.disconnect_serial()
        else:
            self.connect_serial()

    def connect_serial(self):
        port = self.port_name.get().strip()
        if not port:
            messagebox.showwarning("Предупреждение", "Укажите COM-порт!")
            return

        try:
            self.ser = serial.Serial(port, 9600, timeout=0.2)
            self.connected = True
            self.btn_connect.config(text="Отключиться")
            self.btn_ping.config(state=tk.NORMAL)
            self.btn_open.config(state=tk.NORMAL)
            self.btn_clear.config(state=tk.NORMAL)
            self.btn_save.config(state=tk.NORMAL)

            self.status_var.set("ПОДКЛЮЧЕНО (Опрос...)")
            self.lbl_status.config(bg="#0055AA", fg="#FFFFFF")
            self.log_ui("SYS", f"Подключено к порту {port}")

            self.running = True
            self.rx_thread = threading.Thread(target=self.rx_loop, daemon=True)
            self.rx_thread.start()

            self.after(200, self.send_ping)

        except Exception as e:
            messagebox.showerror("Ошибка подключения", str(e))
            self.log_ui("ERR", f"Не удалось открыть порт {port}: {e}")

    def disconnect_serial(self):
        self.running = False
        if self.ser and self.ser.is_open:
```

```
            self.ser.close()
        self.connected = False
        self.btn_connect.config(text="Подключиться")
        self.btn_ping.config(state=tk.DISABLED)
        self.btn_open.config(state=tk.DISABLED)
        self.btn_clear.config(state=tk.DISABLED)
        self.btn_save.config(state=tk.DISABLED)
        self.status_var.set("ОТКЛЮЧЕНО")
        self.lbl_status.config(bg="#333333", fg="#FFFFFF")
        self.log_ui("SYS", "Отключено от порта")

    def send_raw(self, cmd: str):
        if not self.ser or not self.ser.is_open:
            return
        msg = cmd.strip() + "\r\n"
        self.ser.write(msg.encode("utf-8"))
        self.log_ui("TX", msg)

    def send_ping(self):
        self.send_raw("PING")

    def on_key_type(self, event):
        val = self.password_var.get()
        if event.char and event.char.isalnum():
            self.send_raw(event.char)

    def send_auth(self):
        passw = self.password_var.get().strip()
        if not passw:
            messagebox.showwarning("Внимание", "Введите пароль!")
            return
        self.send_raw(f"AUTH:{passw}")
        self.password_var.set("")

    def reset_input(self):
        self.password_var.set("")
        self.send_raw("\r\n")

    def send_new_password(self):
        new_p = self.new_pass_var.get().strip()
        if len(new_p) < 1 or len(new_p) > 16:
            messagebox.showwarning("Внимание", "Длина пароля должна быть от 1 до 16
символов!")
            return
        if not messagebox.askyesno("Подтверждение", f"Записать новый пароль '{new_p}' в
энергонезависимую память EEPROM?"):
            return
        self.send_raw(f"SET:{new_p}")
        self.new_pass_var.set("")

    def rx_loop(self):
        while self.running and self.ser and self.ser.is_open:
            try:
                line = self.ser.readline()
                if line:
                    decoded = line.decode("utf-8", errors="replace").strip()
                    if decoded:
                        self.log_ui("RX", decoded)
                        self.parse_response(decoded)
            except Exception as e:
                self.log_ui("ERR", f"Ошибка чтения: {e}")
                break

    def parse_response(self, text: str):
        if text.startswith("STATUS:"):
            status = text[7:]
            if status == "OK":
                self.status_var.set("[ДОСТУП РАЗРЕШЕН] ЗАМОК ОТКРЫТ")
                self.lbl_status.config(bg="#007700", fg="#FFFFFF")
            elif status == "ERR":
                self.status_var.set("[ОШИБКА] НЕВЕРНЫЙ ПАРОЛЬ")
                self.lbl_status.config(bg="#AA0000", fg="#FFFFFF")
            elif status == "BLOCKED":
```

```
                self.status_var.set("[БЛОКИРОВКА] СИСТЕМА ВРЕМЕННО ЗАБЛОКИРОВАНА")
                self.lbl_status.config(bg="#880000", fg="#FFFFFF")
            elif status == "LOCKED":
                self.status_var.set("ЗАМОК ЗАКРЫТ (ОЖИДАНИЕ ВВОДА)")
                self.lbl_status.config(bg="#333333", fg="#FFFFFF")
            elif status == "SAVED":
                self.status_var.set("[EEPROM] НОВЫЙ ПАРОЛЬ УСПЕШНО СОХРАНЕН")
                self.lbl_status.config(bg="#0055AA", fg="#FFFFFF")
                messagebox.showinfo("EEPROM", "Новый пароль успешно сохранён в
энергонезависимую память!")

        elif text.startswith("DISPLAY:"):
            disp = text[8:]
            self.oled_display_var.set(disp)

        elif text == "PONG":
            self.status_var.set("[СВЯЗЬ УСТАНОВЛЕНА] СТЕНД ГОТОВ К РАБОТЕ")
            self.lbl_status.config(bg="#0055AA", fg="#FFFFFF")

    def on_closing(self):
        self.disconnect_serial()
        self.destroy()

if __name__ == "__main__":
    port = sys.argv[1] if len(sys.argv) > 1 else None
    app = LockClientGUI(port_arg=port)
    app.protocol("WM_DELETE_WINDOW", app.on_closing)
    app.mainloop()
```